

Lista de Exercícios

Estruturas essenciais em C

Questão 1:

Qual é a estrutura correta de um laço **for** em C?

- A) **for**(inicialização; condição; incremento)
- B) **for**(condição; inicialização; incremento)
- C) **for**(incremento; condição; inicialização)
- D) **for**(inicialização; incremento; condição)
- E) **for**(condição; incremento; inicialização)

Questão 2:

Qual é a saída do trecho de código abaixo?

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", i);  
}
```

- A) 1 2 3 4 5
- B) 0 1 2 3 4 5
- C) 1 2 3 4
- D) 0 1 2 3 4
- E) 0 1 2 3

Questão 3:

Sobre a diferença entre **while** e **do-while**, qual afirmação está correta?

- A) O **while** sempre executa o bloco pelo menos uma vez.

- B) O do-while pode nunca executar o bloco.
- C) O for não pode ter uma parte de inicialização.
- D) O while e o do-while são idênticos em todos os casos.
- E) O do-while sempre executa o bloco pelo menos uma vez, pois a condição é verificada depois.

Questão 4:

Qual é a saída do trecho de código abaixo?

```
int x = 1;

while (x <= 5) {
    printf("%d", x);
    x += 2;
}
```

- A) 12345
- B) 1234
- C) 135
- D) 13579
- E) O laço nunca termina

Questão 5:

Qual é a saída do trecho de código abaixo?

```
int i = 10;

do {
    printf("%d ", i);
    i--;
} while (i > 7);
```

- A) 10 9 8 7
- B) 10 9 8

C) 9 8 7

D) 10 9 8 7 6

E) O laço nunca termina

Questão 6:

Quantos asteriscos (*) serão impressos pelo trecho de código abaixo?

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 2; j++) {  
        printf("*");  
    }  
}
```

A) 3

B) 2

C) 5

D) 9

E) 6

Questão 7:

Qual é a saída do trecho de código abaixo?

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    if (i == 5) break;  
    printf("%d", i);  
}
```

A) 124

B) 12345

C) 1245

D) 12

E) 1234

Questão 8:

Qual das alternativas abaixo resultará em um laço infinito?

A) `for (int i = 0; i < 10; i++) { }`

B) `int i = 0; while (i < 10) { i++; }`

C) `int i = 0; do { i++; } while (i < 5);`

D) `int i = 10; while (i > 0) { i++; }`

E) `for (int i = 10; i > 0; i--) { }`

Questão 9:

Qual palavra-chave é utilizada quando uma função não retorna nenhum valor?

A) `int`

B) `void`

C) `return`

D) `null`

E) `empty`

Questão 10:

Qual é a saída do trecho de código abaixo?

```
int soma(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    printf("%d", soma(3, 4));  
    return 0;  
}
```

A) 34

B) 12

C) 7

D) 3

E) 4

Questão 11:

Considerando a função recursiva abaixo, qual é o valor retornado por `fatorial(4)`?

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

A) 4

B) 24

C) 10

D) 16

E) O programa entra em recursão infinita

Questão 12:

Sobre a passagem de parâmetros por valor em C, qual afirmação é verdadeira?

A) A função pode alterar diretamente a variável original passada como argumento.

B) Passagem por valor não é possível em C.

C) Toda função em C utiliza passagem por referência.

D) Parâmetros por valor só funcionam com vetores.

E) A função recebe uma cópia do valor, e alterações dentro dela não afetam a variável original.

Questão 13:

Qual é a saída do trecho de código abaixo?

```
void dobro(int x) {  
    x = x * 2;  
}  
  
int main() {  
    int n = 5;  
    dobro(n);  
    printf("%d", n);  
    return 0;  
}
```

A) 10

B) 0

C) 5

D) Erro de compilação

E) 25

Questão 14:

Considerando a função recursiva abaixo, qual é o valor retornado por fib(5)?

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

A) 5

B) 3

C) 8

D) 4

E) 6

Questão 15:

Qual é a saída do trecho de código abaixo?

```
void zera(int v[], int tam) {  
    for (int i = 0; i < tam; i++) {  
        v[i] = 0;  
    }  
}  
  
int main() {  
    int a[3] = {1, 2, 3};  
    zera(a, 3);  
    printf("%d %d %d", a[0], a[1], a[2]);  
    return 0;  
}
```

A) 1 2 3

B) Erro de compilação

C) 0 2 3

D) 0 0 0

E) 1 0 0

Questão 16:

O que a instrução return faz dentro de uma função?

A) Termina o programa imediatamente, sempre.

B) Só pode ser usada em funções void.

C) Encerra a execução da função e, quando aplicável, devolve um valor ao ponto onde ela foi chamada.

- D) Imprime um valor na tela.
- E) Reinicia a execução da função.

Questão 17:

Em C, o índice do primeiro elemento de um vetor é:

- A) 1
- B) 0
- C) -1
- D) Depende do tipo do vetor
- E) Vetores não possuem índice inicial fixo

Questão 18:

Qual é a saída do trecho de código abaixo?

```
int numeros[5] = {10, 20, 30, 40, 50};  
printf("%d", numeros[2]);
```

- A) 30
- B) 10
- C) 20
- D) 40
- E) 50

Questão 19:

Qual é a saída do trecho de código abaixo?

```
int v[4] = {2, 4, 6, 8};
int soma = 0;

for (int i = 0; i < 4; i++) {
    soma += v[i];
}

printf("%d", soma);
```

- A) 8
- B) 10
- C) 24
- D) 20
- E) 4

Questão 20:

Sobre vetores em C, qual afirmação está correta?

- A) C verifica automaticamente se o índice utilizado está dentro dos limites do vetor.
- B) Acessar um índice fora dos limites de um vetor é seguro e não causa nenhum problema.
- C) Vetores em C sempre começam no índice 1.
- D) O tamanho de um vetor pode ser alterado automaticamente durante a execução do programa.
- E) C não realiza verificação automática de limites; acessar um índice inválido pode causar comportamento indefinido.

Questão 21:

Qual é a saída do trecho de código abaixo?

```
int v[3] = {5, 10, 15};  
v[1] = v[1] + v[2];  
printf("%d", v[1]);
```

A) 10

B) 15

C) 5

D) 25

E) 30

Questão 22:

Qual é a saída do trecho de código abaixo?

```
int v[5] = {3, 7, 2, 9, 4};  
int maior = v[0];  
  
for (int i = 1; i < 5; i++) {  
    if (v[i] > maior) {  
        maior = v[i];  
    }  
}  
  
printf("%d", maior);
```

A) 3

B) 7

C) 9

D) 4

E) 5

Questão 23:

Qual é a saída do trecho de código abaixo?

```
int buscar(int v[], int tam, int valor) {  
    for (int i = 0; i < tam; i++) {  
        if (v[i] == valor) return i;  
    }  
    return -1;  
}  
  
int main() {  
    int a[4] = {10, 20, 30, 40};  
    printf("%d", buscar(a, 4, 30));  
    return 0;  
}
```

- A) -1
- B) 1
- C) 3
- D) 30
- E) 2

Questão 24:

Qual é a principal função de uma struct em C?

- A) Permitir armazenar apenas um tipo de dado por vez.
- B) Agrupar diferentes tipos de dados em uma única estrutura, criando um novo tipo personalizado.
- C) Armazenar apenas números.
- D) Substituir totalmente a necessidade de variáveis comuns.
- E) Funcionar apenas dentro de funções void.

Questão 25:

Qual operador é utilizado para acessar um campo de uma variável do tipo struct (não ponteiro)?

- A) .
- B) ->
- C) ::
- D) &
- E) []

Questão 26:

Qual é a saída do trecho de código abaixo?

```
struct Ponto {  
    int x;  
    int y;  
};  
  
struct Ponto p1 = {3, 7};  
printf("%d", p1.x + p1.y);
```

- A) 10
- B) 3
- C) 7
- D) 37
- E) 73

Questão 27:

Qual é a principal vantagem de usar typedef ao declarar uma struct?

- A) Permitir usar índices na struct como em um vetor.
- B) Tornar os campos da struct privados.

- C) Fazer a struct ocupar menos espaço na memória.
- D) Evitar a necessidade de escrever a palavra struct toda vez que uma variável desse tipo for declarada.
- E) Permitir que a struct tenha herança, como em POO.

Questão 28:

Qual é a saída do trecho de código abaixo?

```
typedef struct {  
    char nome[10];  
    int idade;  
} Pessoa;  
  
Pessoa time[2] = {  
    {"Lucas", 20},  
    {"Marina", 25}  
};  
  
printf("%d", time[1].idade);
```

- A) 20
- B) 10
- C) 2
- D) Erro de compilação
- E) 25

Questão 29:

Considerando uma struct Aluno já declarada, com um campo char nome[50], e uma variável aluno1 desse tipo já declarada, qual é a forma correta de atribuir o texto "Maria" a esse campo?

- A) aluno1.nome = "Maria";
- B) aluno1->nome = "Maria";

C) strcpy(aluno1.nome, "Maria");

D) nome(aluno1) = "Maria";

E) aluno1.nome() = "Maria";

Questão 30:

Considerando as structs abaixo:

```
struct Endereco {  
    char cidade[20];  
};
```

```
struct Pessoa {  
    char nome[20];  
    struct Endereco end;  
};
```

```
struct Pessoa p;  
strcpy(p.end.cidade, "Recife");
```

Qual é a forma correta de acessar o campo cidade, dentro de Endereco, a partir da variável p?

A) p.cidade

B) p.end.cidade

C) p->end->cidade

D) p[end].cidade

E) end.p.cidade

Gabarito:

1-A | 2-D | 3-E | 4-C | 5-B | 6-E | 7-A | 8-D | 9-B | 10-C | 11-B | 12-E | 13-C |
14-A | 15-D | 16-C | 17-B | 18-A | 19-D | 20-E | 21-D | 22-C | 23-E | 24-B |
25-A | 26-A | 27-D | 28-E | 29-C | 30-B